

Motores de Plantillas en Express

GUÍA DE CONCEPTOS DETALLADOS - UNIDAD 3

1. Introducción a los Motores de Plantillas

Los motores de plantillas son herramientas que permiten generar HTML dinámico desde el servidor. Dado que Express.js no incluye un motor por defecto, nos da la libertad de elegir e integrar opciones como **Pug** (anteriormente Jade) o **EJS** (Embedded JavaScript).

¿Para qué sirven?

- Insertar datos dinámicos provenientes del backend hacia las vistas.
- Reutilizar componentes (como headers o footers) y layouts completos.
- Implementar lógica (condicionales y bucles) directamente en el HTML.
- Mantener el código organizado y modular.

2. Configuración en Express

Para integrar un motor de plantillas en Express, necesitamos instalarlo y configurarlo indicando qué motor usaremos y dónde estarán ubicados los archivos de vista.

```
const express = require('express');
const path = require('path');
const app = express();

// Configuración del motor (ejemplo con EJS)
app.set('view engine', 'ejs');
// Definir la carpeta donde vivirán las vistas
app.set('views', path.join(__dirname, 'views'));

// Renderizando una vista
```

```
app.get('/', (req, res) => {
  res.render('index', {
    title: 'Mi aplicación Express',
    message: '¡Hola desde el servidor!'
  });
});
```

3. Pug (Anteriormente Jade)

Pug es un motor de plantillas de alto rendimiento con una sintaxis concisa. Su mayor característica es que **elimina las etiquetas de cierre** y utiliza la **indentación estricta** para definir la jerarquía del HTML.

Características de Pug

- **Sintaxis Limpia:** Los atributos van entre paréntesis `a(href="/")` y el texto va después de un espacio.
- **Interpolación:** Los datos se inyectan usando `#{variable}` o el signo de igual `h1= message`.
- **Herencia (Layouts):** Permite definir una plantilla base con `block content` y luego extenderla en otras vistas usando `extends layout`.
- **Mixins:** Funciones integradas para crear bloques de código reutilizables (ej. tarjetas de productos).

4. EJS (Embedded JavaScript)

EJS es ideal para quienes vienen del desarrollo Frontend puro, ya que utiliza HTML tradicional y simplemente permite incrustar etiquetas especiales para ejecutar código JavaScript.

Etiquetas principales en EJS

- `<% %>` : Ejecuta lógica JavaScript (if, for, while) sin mostrar nada en pantalla.
- `<%= %>` : Muestra el valor de una variable en el HTML (escapando los caracteres para seguridad).
- `<%- %>` : Muestra el valor de una variable **sin escapar** (útil para inyectar HTML procesado o incluir partials).
- `<%- include('partials/header') %>` : Permite reutilizar fragmentos de código.

5. Comparativa: Pug vs EJS

Característica	Pug	EJS
Sintaxis	Basada en indentación, sin etiquetas de cierre.	HTML tradicional con etiquetas de incrustación.
Curva de Aprendizaje	Más pronunciada si estás acostumbrado a HTML.	Muy rápida y natural.
Modularidad	Sistema de bloques (Layouts) y Mixins integrados.	Requiere <code>include()</code> o librerías extra (express-ejs-layouts).
Errores comunes	Sensibilidad extrema a los espacios vs tabulaciones.	Olvidar cerrar corchetes de JavaScript <code>} %></code> .

6. Buenas Prácticas y Seguridad

- **Separación de Responsabilidades:** Mantén la lógica de negocio, consultas y validaciones en los *Controladores*, dejando que las *Vistas* (Pug/EJS) solo se encarguen de mostrar los datos.
- **Principio DRY (Don't Repeat Yourself):** Evita copiar y pegar el menú de navegación en cada archivo. Usa Layouts (Pug) o Partials (EJS).
- **Validación y Formularios:** Maneja los errores de formulario desde el backend y pasa el array de errores a la plantilla para mostrarlos dinámicamente.
- **Prevención de XSS (Cross-Site Scripting):** Tanto Pug como EJS escapan automáticamente el texto para evitar la inyección de scripts maliciosos. Utiliza las versiones "sin escapar" (`!=` en Pug, `<%-` en EJS) **solo** cuando confíes plenamente en la fuente del dato.